



DATA STRUCTURES AND ALGORITHMS

23CSE203

LAB MANUAL

Submitted by	
Name	PRANAVI AYYAGARI
Roll No	AV.SC.U4CSE25012
Year/Sem/Section	2 nd year/Sem-3/CSE-A
Credits	05

Submitted to	
Name	Dr. RAJ KUMAR BATCHU
Department	School of Computing

Designation	Asst. Professor (SI Grade)
-------------	----------------------------

S.NO	TITLE	PAGE NO	REMARKS
1.	WEEK 1: 1a. Recursive countdown from n to Launch. 1b. Recursive calculation of compound growth factor. 1c. Recursive search for employee ID in list. 1d. Recursive factorial calculation for parcel arrangements. 1e. Recursive Fibonacci series generation.	3 - 11	
2.	WEEK 2:		
3.	WEEK 3:		
4.	WEEK 4:		

WEEK – 1

1a. AIM: A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursive. Write a recursive python program to display a countdown from n to 1, followed by “Launch”.

PROGRAM:

```
pract.py - C:/Users/pranavi/OneDrive/Desktop/DSA PRACTICE/pract.py (3.9.1)
File Edit Format Run Options Window Help
def countdown(n):
    if n <= 0:
        print("Rocket is Launched 🚀")
    else:
        print(n)
        countdown(n - 1)

print(countdown(10))
```

OUTPUT:

Positive Case;

```
==== RESTART: C:/Users/pranavi/OneDrive/Desktop/DSA PRACTICE/pract.py =====
10
9
8
7
6
5
4
3
2
1
Rocket is Launched 🚀
None
>>> |
```

Negative Case;

```
Rocket is Launched 🚀
None
```

EXPLANATION:

Condition 1: Positive Input ($n > 0$)

- The function checks if $n \leq 0$. Since n is positive, this condition is False.
- It executes the else block:
 - Prints the current value of n .

- Calls itself recursively with $n - 1$.
- This repeats until n becomes 0.
- Finally, when $n = 0$, the condition $n \leq 0$ becomes True.
- After the recursive calls finish, the function returns None (since no return is defined).
- The calling print (countdown (10)) will display the countdown, the launch message, and then None.

Condition 2: Negative Input ($n \leq 0$)

- The condition if $n \leq 0$ is immediately **True**.
- It directly prints:

Rocket is Launched
- No recursion happens because the else block is skipped.
- The function ends and returns None.
- The calling print (countdown (-1)) will display:

Rocket is Launched

None

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Syntax Error Forgot to give ":" in the control statement.	Later it was rectified by giving the ":"
2.	Type Error I have entered a non – integer.	It was rectified by passing only the integers.

1b. AIM: Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n (power), Where p = principal growth factor, n = number of years.

PROGRAM:

```
pract.py - C:/Users/pranavi/OneDrive/Desktop/DSA PRACTICE/pract.py (3.9.1)
File Edit Format Run Options Window Help
p = float(input("Enter the Principal Growth Factor: "))
n = int(input("Enter the Number of Years: "))

def power(p, n):
    if n == 0:
        return 1
    else:
        return p * power(p, n - 1)

Output = power(p, n)
print("Growth of an Investment", Output)
```

OUTPUT:

Positive Case;

```
>>>
===== RESTART: C:/Users/pranavi/OneDriv
Enter the Principal Growth Factor: 34
Enter the Number of Years: 5
Growth of an Investment 45435424.0
>>>
```

Negative Case;

```
Enter the Principal Growth Factor:14
Enter the Number of Years:-5
Traceback (most recent call last):
  File "C:/Users/M.Hansini Sarora/Downloads/Compound Interest.py", line 8, in <module>
    Output = power(p,n)
  File "C:/Users/M.Hansini Sarora/Downloads/Compound Interest.py", line 7, in power
    return p * power(p, n-1)
  File "C:/Users/M.Hansini Sarora/Downloads/Compound Interest.py", line 7, in power
    return p * power(p, n-1)
  File "C:/Users/M.Hansini Sarora/Downloads/Compound Interest.py", line 7, in power
    return p * power(p, n-1)
  [Previous line repeated 1023 more times]
RecursionError: maximum recursion depth exceeded
>>>
```

EXPLANATION:

Condition 1: Positive Input ($n \geq 0$)

- User enters a positive number of years (e.g., $n = 3$).
- The function checks if $n == 0$. Since n is positive, this is False.
- It executes the else block:
 - Multiplies p by the result of $\text{power}(p, n-1)$.
- Recursion continues until n becomes 0.
- At $n = 0$, the base case returns 1.

- The recursive calls unwind, multiplying p repeatedly.
- The output is p^n , i.e., the growth factor raised to the number of years.

Condition 2: Negative Input ($n < 0$)

- User enters a negative number of years (e.g., $n = -2$).
- The function checks if $n == 0$. Since n is negative, this is **False**.
- It executes the else block:
 - Calls `power(p, n-1)` with an even smaller negative number.
- Recursion continues indefinitely ($-2 \rightarrow -3 \rightarrow -4 \rightarrow \dots$).
- The base case ($n == 0$) is never reached.
- Python raises a “Recursion Error: maximum recursion depth exceeded.”

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Floating-Point I have entered a floating-point value in the years.	Later it was rectified by giving the integer value.
2.	Large Input Error I have given a huge input value.	It was rectified by passing default limit of Python.

1c. AIM: An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

PROGRAM:

```
pract.py - C:/Users/pranavi/OneDrive/Desktop/DSA PRACTICE/pract.py (3.9.1)
File Edit Format Run Options Window Help
def employee(emp_list, target, index=0):
    if index == len(emp_list):
        return False

    if emp_list[index] == target:
        return True

    return employee(emp_list, target, index + 1)

employees = [510, 511, 512, 513, 514]
target_id = int(input("Enter Employee ID to search: "))

if employee(employees, target_id):
    print("Employee ID Found ✓")
else:
    print("Employee ID Not Found ✗")
```

OUTPUT:

Positive Case;

```
===== RESTART: C:/Users/pranavi/OneDrive/Desktop/DSA PRACTICE/pract.py =====
Enter Employee ID to search: 510
Employee ID Found ✓
>>>
```

Negative Case;

```
>>>
===== RESTART: C:/Users/pranavi/OneDrive/Desktop/DSA PRACTICE/pract.py =====
Enter Employee ID to search: 45
Employee ID Not Found ✗
>>>
```

EXPLANATION:

Condition 1: Employee ID Found

- The function starts at index 0.
- It compares `emp_list[index]` with the target.
- If a match is found, the function immediately returns True.
- The recursion stops early — no need to check further elements.
- The main program prints:

Employee ID Found ✓

Condition 2: Employee ID Not Found

- The function checks each element recursively.

- If the end of the list is reached (`index == len(emp_list)`), the base case returns False.
- The recursion unwinds, propagating False back to the main program.
- The main program prints:

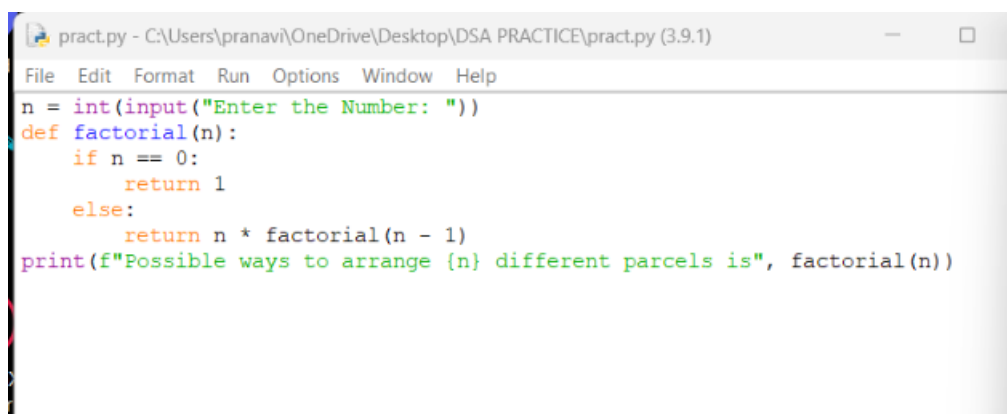
Employee ID Not Found ✕

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Empty List I haven't entered the values in the list.	The empty list was filled with the emp IDs.
2.	Logical Error I have entered the duplicate values.	It was rectified by removing the duplicates.

1d. AIM: In a logistics company, the number of possible ways to arrange n different parcels is calculated using the factorial formula ($n!$). Develop a recursive program to calculate the factorial of a given number.

PROGRAM:



```
pract.py - C:\Users\pranavi\OneDrive\Desktop\DSA PRACTICE\pract.py (3.9.1)
File Edit Format Run Options Window Help
n = int(input("Enter the Number: "))
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)
print(f"Possible ways to arrange {n} different parcels is", factorial(n))
```

OUTPUT:

Positive Case;


```

===== RESTART: C:\Users\pranavi\OneDrive\Desktop\DSA PRACTICE\pract.py =====
Enter the Number: 21
Possible ways to arrange 21 different parcels is 51090942171709440000
'''

```

Negative Case;

```

Enter the Number:-58
Traceback (most recent call last):
  File "C:\Users\M.Hansini Sarora\Downloads\Factorial.py", line 7, in <module>
    print(f"Possible ways to arrange (n) different parcels is",factorial(n))
  File "C:\Users\M.Hansini Sarora\Downloads\Factorial.py", line 6, in factorial
    return n * factorial(n - 1)
  File "C:\Users\M.Hansini Sarora\Downloads\Factorial.py", line 6, in factorial
    return n * factorial(n - 1)
  File "C:\Users\M.Hansini Sarora\Downloads\Factorial.py", line 6, in factorial
    return n * factorial(n - 1)
  [Previous line repeated 1023 more times]
RecursionError: maximum recursion depth exceeded
>>>

```

EXPLANATION:

Condition 1: When $n == 0$

- The function checks:

if $n == 0$:

return 1

- By definition, $0! = 1$.
- So if the user enters 0, the program outputs:

Possible ways to arrange 0 different parcels is 1

Condition 2: When $n > 0$

- The function executes:

else:

return $n * \text{factorial}(n - 1)$

- This means the function calls itself with a smaller number until it eventually reaches $n == 0$.
- Uses recursion to multiply numbers down to 1.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Negative Input	Later, It was rectified by entering positive values.

	I haven't entered the negative value.	
2.	Large Input Error I have entered a very large number.	It was rectified by entering limited values.

1e. AIM: Write a recursive python program to print the first n terms of the Fibonacci Series.

PROGRAM:

```
pract.py - C:\Users\pranavi\OneDrive\Desktop\DSA PRACTICE\pract.py (3.9.1)
File Edit Format Run Options Window Help
n = int(input("Enter the number: "))
def fib(n):
    if n <= 1:
        return n
    else:
        return fib(n - 1) + fib(n - 2)

for i in range(n):
    print(fib(i), end=" ")
|
```

OUTPUT:

Positive Case;

```
===== RESTART: C:\Users\pranavi\OneDrive\Desktop\DSA PRACTICE\pract.py =====
Enter the number: 7
0 1 1 2 3 5 8
>>>
```

Negative Case;

```
>>> Enter the number:-5
```

EXPLANATION:

Condition 1: When $n \leq 1$

- Inside the function:

if $n \leq 1$:

return n

- This is the **base case** of recursion.
- If $n = 0$, it returns 0.
- If $n = 1$, it returns 1.
- These are the starting points of the Fibonacci sequence..

Condition 2: When $n > 1$

- The function executes:

python

else:

return fib($n - 1$) + fib($n - 2$)

- This means the Fibonacci number at position n is the **sum of the two previous numbers**.
- The recursion keeps breaking down until it reaches the base case ($n \leq 1$).

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Negative Input I haven't entered the negative value.	Later, it was rectified by entering positive values.
2.	Very large input Recursion depth exceeded.	It was rectified by entering limited values.

WEEK-2

A) AIM: Write a python program to perform linear search on the given input.

Program:

```
pract.py - C:\Users\pranavi\OneDrive\Desktop\DSA PRACTICE\pract.py (3.9.1)
File Edit Format Run Options Window Help
#Linear search

n=int(input("Enter no.of elements:"))
arr=[]
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
key=int(input("enter the number:"))
def linear_search(arr,key):
    for i in range(0,len(arr)):
        if arr[i]==key:
            print("Element found at Index:",i)
            return
    print("Not found")
linear_search(arr,key)
```

Output:

```
IDLE Shell 3.12.0
File Edit Shell Debug Options Window Help
Python 3.12.0 (tags/v3.12.0:0fb18b0, Oct 2 2023, 13:03:39) [MSC v.1935 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> = RESTART: C:\Users\CSE-Lab1\Desktop\dsa 25029\linear andbinary search.py
Enter no.of elements:6
Enter the elements:
1
3
5
7
4
3
enter the number:4
Element found at Index: 4
>>>
```

Explanation:

- In linear search we can append elements in sorted or unsorted way also.
- After placing the target we need to check the all the elements in a order.
- The time complixity is $O(n)$.
- The target element may be found at beginning or last or at middle.

Error table:

s.no	Error obtained	Rectification
------	----------------	---------------

1.	Return placed in wrong place	Corrected later
2.	Calling the function	Rectified

B) Aim: Write a python program to perform binary search on the list.

Program: Unsorted

```

#binary search unsorted

n=int(input("Enter the o.of elements:"))
arr=[]
print("Enter the element:")
for i in range(n):
    arr.append(int(input()))
arr.sort()
print("sorted:",arr)
key=int(input("Enter the target:"))
def binary_search(arr,key):
    low=0
    high=len(arr)-1
    while low<=high:
        mid=(low+high)//2
        if arr[mid]==key:
            return mid
        elif arr[mid]<key:
            low=mid+1
        else:
            high=mid-1
    return -1
print(binary_search(arr,key))

```

Output:

```

IDLE Shell 3.12.0
Python 3.12.0 (tags/v3.12.0:0fb18b0, Oct 2 2023, 13:03:39) [MSC v.1935 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> = RESTART: C:\Users\CSE-Lab1\Desktop\dsa 25029\linear andbinary search.py
Enter the o.of elements:5
Enter the element:
1
5
7
9
3
sorted: [1, 3, 5, 7, 9]
Enter the target:4
-1

```

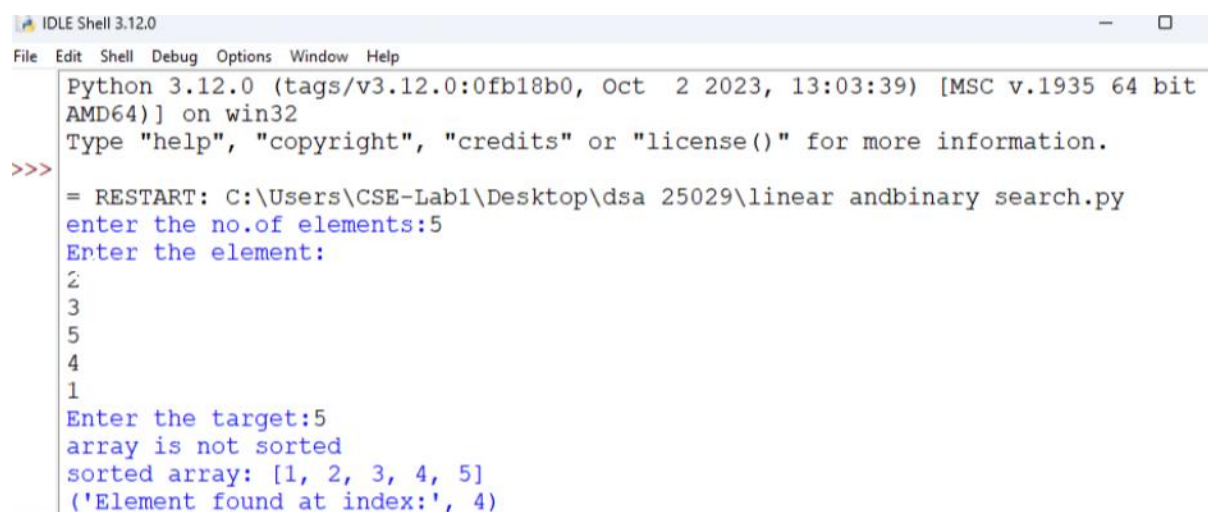
Program: Sorted

```

#binary search sorted
n=int(input("enter the no.of elements:"))
arr=[]
print("Enter the element:")
for i in range(n):
    arr.append(int(input()))
key=int(input("Enter the target:"))
if arr==sorted(arr):
    print("It is sorted")
else:
    print("array is not sorted")
    arr.sort()
    print("sorted array:",arr)
def binary_search(arr,key):
    low=0
    high=len(arr)-1
    while low<=high:
        mid=(low+high)//2
        if arr[mid]==key:
            return 'Element found at index:', mid
        elif arr[mid]<key:
            low=mid+1
        else:
            high=mid-1
    return -1
print(binary_search(arr,key))

```

Output:



```

IDLE Shell 3.12.0
File Edit Shell Debug Options Window Help
Python 3.12.0 (tags/v3.12.0:0fb18b0, Oct 2 2023, 13:03:39) [MSC v.1935 64 bit
AMD64] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> = RESTART: C:\Users\CSE-Lab1\Desktop\dsa 25029\linear andbinary search.py
enter the no.of elements:5
Enter the element:
2
3
5
4
1
Enter the target:5
array is not sorted
sorted array: [1, 2, 3, 4, 5]
('Element found at index:', 4)

```

Explanation:

- The binary search should always executed in sorted order.
- Either it can be ascending or descending order.

- So the program is in unsorted order so we need to do sorting first later the binary search is executed.
- Error table:

s.no	Error obtained	Rectification
1.	Mistakes in main logic	Rectified
2.	Errors in conditions	Rectified

WEEK-3

A)Aim: Write a python programme to sort the elements using a bubble sort technique. Take an input elements from the users.

Programme:

```
3.py - C:/Users/pranavi/OneDrive/Desktop/DSA PRACTICE/3.py (3.9.1)
File Edit Format Run Options Window Help
# Bubble Sort
n = int(input("Enter the number of elements: "))
arr = []
for i in range(n):
    x = int(input(f"Enter element {i + 1}: "))
    arr.append(x)
for i in range(n - 1):
    for j in range(n - i - 1):
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]
print("Sorted array:", arr)
```

Output:

```
===== RESTART: C:/Users/pranavi/OneDrive/Desktop/DSA PRACTICE/3.py =====
Enter the number of elements: 5
Enter element 1: 12
Enter element 2: 9
Enter element 3: 45
Enter element 4: 27
Enter element 5: 100
Sorted array: [9, 12, 27, 45, 100]
>>> |

Enter the number of elements: 0
Sorted array: []
>>> |
```

Explanation:

- n stores the number of elements, and an empty list arr is created.
- The first for loop takes n numbers from the user and adds them to arr.
- The second loop controls how many times the sorting process is repeated.

- The inner loop compares two neighboring elements: `arr[j]` and `arr[j+1]`.
- If the first element is bigger, they are swapped using `arr[j], arr[j+1] = arr[j+1], arr[j]`.
- Finally, `print()` displays the array in ascending order.

Error table:

s.no	Error obtained	Rectification
1.	Mistakes in main logic	Rectified
2.	Errors in conditions	Rectified

B)Aim: Write a python programme to sort the elements using a insertion sort technique.
Take an input elements from the users.

Programme:

3.py - C:/Users/pranavi/OneDrive/Desktop/DSA PRACTICE/3.py (3.9.1)

File Edit Format Run Options Window Help

```
#insertion sort

def insertion_sort(arr):
    n=len(arr)
    for j in range(1,n):
        key=arr[j]
        j=j-1
        while j>=0 and arr[j]>key:
            arr[j+1]=arr[j]
            j=j-1
        arr[j+1]=key
    return arr
n=int(input("enter the no.of elements:"))
arr=[]
for i in range(n):
    x=int(input(f"Enter the number{i+1}:"))
    arr.append(x)
insertion_sort(arr)
print("sorted array:",arr)
```

Output:

```
===== RESTART: C:/Users/pranavi/OneDrive/Desktop/DSA PRACTICE/3.py =====
enter the no.of elements:5
Enter the number1:12
Enter the number2:23
Enter the number3:45
Enter the number4:76
Enter the number5:100
sorted array: [12, 23, 45, 76, 100]
>>> |
```

```
-
| enter the no.of elements:0
| sorted array: []
>>> |
```

Explanation:

- insertion_sort(arr) takes the array and finds its length using $n = \text{len}(\text{arr})$.
- The for loop starts from the second element because the first element is considered already sorted.
- $\text{key} = \text{arr}[j]$ stores the current element that we want to place in the correct position.
- The while loop moves bigger elements one position to the right.

- `arr[j+1] = key` places the current element in its correct position.
- Finally, the function returns the sorted array, and `print()` displays it in ascending order.

Error table:

S.no	Errors	Rectification
1.	Error in main logic	Rectified
2.	Errors in condition	rectified

C)Aim: Write a python programme to sort the elements using a selection sort technique.
Take an input elements from the users.

Programme:

```
3.py - C:/Users/pranavi/OneDrive/Desktop/DSA PRACTICE/3.py (3.9.1)
File Edit Format Run Options Window Help
def selection_sort(arr):
    n=len(arr)
    for i in range(n-1):
        min_index=i
        for j in range(i+1,n):
            if arr[j]<arr[min_index]:
                min_index=j
        arr[i],arr[min_index]=arr[min_index],arr[i]
    return arr
n=int(input("enter the no.of elements:"))
arr=[]
for i in range(n):
    x=int(input(f"Enter the number{i+1}:"))
    arr.append(x)
selection_sort(arr)
print("sorted array:",arr)
```

Output:

```
===== RESTART: C:/Users/pranavi/OneDrive/Desktop/DSA PRACTICE/3.py =====
enter the no.of elements:5
Enter the number1:23
Enter the number2:18
Enter the number3:26
Enter the number4:19
Enter the number5:2
sorted array: [2, 18, 19, 23, 26]
>>>

>> | enter the no.of elements:0
    | sorted array: []
```

Explanation:

- `n = len(arr)` finds the total number of elements in the array.
- The outer for loop selects each position where the smallest element should be placed.

- `min_index = i` assumes that the current position contains the smallest element.
- The inner for loop compares the remaining elements and finds the smallest element's index.
- `arr[i], arr[min_index] = arr[min_index], arr[i]` swaps the smallest element with the current element.
- After all iterations, the array is sorted in ascending order and `print()` displays it.

Error table:

S.no	Errors	Rectification
1.	Error in main logic	Rectified
2.	Errors in condition	rectified

D)Aim: Write a python programme to sort the elements using a merge sort technique. Take an input elements from the users.

Programme:

3.py - C:/Users/pranavi/OneDrive/Desktop/DSA PRACTICE/3.py (3.9.1)

```
File Edit Format Run Options Window Help
def merge_sort(arr):
    if len(arr)<=1:
        return arr
    mid=len(arr)//2
    left=arr[:mid]
    right=arr[mid:]
    merge_sort(left)
    merge_sort(right)
    i=j=k=0
    while i<len(left) and j<len(right):
        if left[i]<=right[j]:
            arr[k]=left[i]
            i=i+1
        else:
            arr[k]=right[j]
            j=j+1
        k=k+1
    while i<len(left):
        arr[k]=left[i]
        i=i+1
        k=k+1
    while j<len(right):
        arr[k]=right[j]
        j=j+1
        k=k+1
n=int(input("enter the no.of elements:"))
arr=[]
for i in range(n):
    x=int(input(f"Enter the number{i+1}:"))
    arr.append(x)
merge_sort(arr)
print("sorted array:",arr)
```

Output:

```
enter the no.of elements:0
sorted array: []
.>> |
```

```
===== RESTART: C:/Users/pranavi/OneDrive/Desktop/DSA PRACTICE/3.py =====
enter the no.of elements:6
Enter the number1:12
Enter the number2:17
Enter the number3:29
Enter the number4:45
Enter the number5:98
Enter the number6:78
sorted array: [12, 17, 29, 45, 78, 98]
```

Explanation:

- if `len(arr) <= 1` checks if the array has 0 or 1 element; if yes, it is already sorted.
- `mid = len(arr)//2` finds the middle and divides the array into left and right parts.
- `merge_sort(left)` and `merge_sort(right)` recursively divide both parts into smaller parts.
- `i = j = k = 0` creates indexes to compare elements from the left part, right part, and original array.
- The while loops compare elements and put the smaller element first into the original array.
- Finally, the remaining elements are added, and `print()` displays the sorted array in ascending order.

Error table:

S.no	Errors	Rectification
1.	Error in main logic	Rectified
2.	Errors in condition	rectified

E)Aim: Write a python programme to sort the elements using a merge sort technique. Take an input elements from the users.

Programme:


```
3.py - C:/Users/pranavi/OneDrive/Desktop/DSA PRACTICE/3.py (3.9.1)
File Edit Format Run Options Window Help
def quick_sort(a,low,high):
    if low<high:
        i=low
        j=high
        pivot=low
        while i<j:
            while i<len(a) and a[i]<=a[pivot]:
                i=i+1
            while a[j]>a[pivot]:
                j=j-1
            if i<j:
                a[i],a[j]=a[j],a[i]
            a[j],a[pivot]=a[pivot],a[j]
            quick_sort(a,low,j-1)
            quick_sort(a,j+1,high)
        return a
n=int(input("Enter the no.of elements:"))
a=[]
for i in range(n):
    x=int(input(f"enter the number{i+1}:"))
    a.append(x)
quick_sort(a,0,n-1)
print("sorted array:",a)
```

Output:

```
===== RESTART: C:/Users/pranavi/OneDrive/Desktop/DSA PRACTICE/3.py =====
Enter the no.of elements:5
enter the number1:13
enter the number2:5566
enter the number3:67
enter the number4:90
enter the number5:12
sorted array: [12, 13, 67, 90, 5566]
...
| enter the no.of elements:0
| sorted array: []
.>> |
```

Explanation:

- quick_sort(a, low, high) sorts the elements between the low and high indexes.
- pivot = low selects the first element as the pivot.
- i moves from the left and finds an element greater than the pivot, while j moves from the right and finds an element smaller than or equal to the pivot.
- If $i < j$, $a[i]$ and $a[j]$ are swapped.

- After the loop, `a[j]` and the pivot are swapped, putting the pivot in its correct position.
- `quick_sort()` is called again for the left and right parts, and finally the array is printed in sorted order.

Error table:

S.no	Errors	Rectification
1.	Error in main logic	Rectified
2.	Errors in condition	rectified